



**МИНИСТЕРСТВО ОБРАЗОВАНИЯ, НАУКИ И МОЛОДЕЖИ
РЕСПУБЛИКИ КРЫМ**

**Государственное бюджетное образовательное учреждение высшего
образования Республики Крым
«Крымский инженерно-педагогический университет имени Февзи Якубова»
(ГБОУВО РК КИПУ имени Февзи Якубова)**

**Факультет экономики, менеджмента и информационных технологий
Кафедра прикладной информатики**

**Направление подготовки 09.03.03 Прикладная информатика
Профиль «Прикладная информатика в информационной сфере»**

КУРСОВОЙ ПРОЕКТ

**по дисциплине «Разработка мобильных приложений»
на тему**

**«Мобильное приложение для отслеживания физической активности с
использованием встроенных датчиков»**

отлично
16.05.2025
[Signature]

**Выполнил:
Студент III курса
группы И-2-22
очной формы обучения
Куриленко Максим Дмитриевич**

[Signature]
(подпись)

**Научный руководитель:
к.т.н., доц. Абдурайимов Л.Н.**

[Signature]
(подпись)

Симферополь – 2025

АННОТАЦИЯ

Куриленко М.Д. Мобильное приложение для отслеживания физической активности с использованием встроенных датчиков.

В работе представлена технология разработки мобильного приложения, ориентированного на мониторинг физической активности пользователя посредством встроенных датчиков смартфона. Впервые предложено решение, объединяющее анализ данных акселерометра, гироскопа и GPS для точного подсчёта шагов, расстояния и расхода калорий с возможностью интеграции с облачными сервисами. Приложение разработано с использованием клиент-серверной архитектуры, обеспечивающей синхронизацию данных между устройствами и формирование персональных рекомендаций. Программный продукт обладает интуитивным интерфейсом и может быть использован как инструмент для поддержания здорового образа жизни.

Ключевые слова: мобильное приложение, физическая активность, встроенные датчики, акселерометр, GPS, клиент-серверная архитектура, Kotlin, Android, Firebase.

ABSTRACT

Kurylenko M.D. Mobile application for tracking physical activity using built-in sensors.

The paper presents the technology of developing a mobile application focused on monitoring user physical activity via built-in smartphone sensors. For the first time, a solution combining accelerometer, gyroscope, and GPS data analysis is proposed to accurately count steps, distance, and calorie expenditure, with cloud service integration. The application is developed using a client-server architecture for data synchronization and personalized recommendations. The software features an intuitive interface and serves as a tool for promoting a healthy lifestyle.

Key words: mobile application, physical activity, built-in sensors, accelerometer, GPS, client-server architecture, Kotlin, Android, Firebase.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ МОБИЛЬНЫХ ПРИЛОЖЕНИЙ ДЛЯ МОНИТОРИНГА ФИЗИЧЕСКОЙ АКТИВНОСТИ.....	7
1.1. Общая информация о мобильных приложениях и их категориях.	7
1.2. Сравнительный анализ существующих решений.....	8
1.3. Анализ технологий и языков программирования.....	14
1.4. Основные этапы разработки.	17
1.5. Выбор платформы и инструментов.....	22
Выводы по первой главе.....	26
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ FITTRACK	28
2.1. Проектирование архитектуры приложения	28
2.2. Реализация сервисного слоя и интеграция с API.....	32
2.3. Разработка модуля обработки данных с датчиков и сервисов.....	36
2.4. Реализация локального хранения данных	40
2.5. Оптимизация производительности и многопоточная обработка.....	44
2.6. Тестирование, сборка и развертывание приложения.	45
Выводы по второй главе.....	47
ЗАКЛЮЧЕНИЕ	48
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	49
ПРИЛОЖЕНИЕ А	51

ВВЕДЕНИЕ

Актуальность темы. В современном мире, где здоровый образ жизни и цифровые технологии становятся неотъемлемой частью повседневности, мобильные приложения для мониторинга физической активности приобретают особую значимость. По данным исследований, более 60% владельцев смартфонов используют фитнес-приложения, однако многие сталкиваются с рядом проблем: ограниченным функционалом, необходимостью платных подписок, недостаточной точностью или отсутствием интеграции с другими сервисами. Актуальность разработки приложения FitTrack обусловлена следующими факторами: Глобальный рынок фитнес-приложений демонстрирует стремительный рост и, по прогнозам, достигнет 23,4 миллиарда долларов к 2026 году с темпом роста 21,6%. Пандемия COVID-19 ускорила эту тенденцию, переориентировав пользователей на домашние тренировки с использованием мобильных технологий. Повышение осведомленности о важности здорового образа жизни привело к формированию культуры самоконтроля здоровья и запросу на персонализированные рекомендации. Социальные аспекты фитнес-приложений способствуют формированию устойчивой мотивации пользователей. Развитие технологий ИИ и машинного обучения открывает новые возможности для анализа данных, а интеграция с носимыми устройствами повышает точность измерений. Происходит трансформация подхода к здравоохранению от лечения к профилактике. Исследования показывают, что использование фитнес-приложений может снизить риск развития сердечно-сосудистых заболеваний и диабета. Геймификация и социальные механики повышают мотивацию пользователей: люди, использующие такие приложения, на 32% чаще придерживаются регулярных тренировок. Рост популярности активного транспорта способствует снижению углеродного следа, делая фитнес-приложения частью тренда на устойчивое развитие. Таким образом, разработка приложения, использующего

встроенные датчики смартфона и взаимодействующего с внешними API, позволяет создать универсальное и доступное решение, отвечающее современным потребностям в области здоровья и технологий.

Целью курсового проекта является разработка мобильного приложения для мониторинга физической активности с использованием встроенных датчиков, клиент-серверной архитектуры и современных технологий Android-разработки.

Задачи исследования:

1. Провести анализ существующих аналогов и выявить их ключевые недостатки.
2. Изучить технологии работы с датчиками смартфона и выбрать оптимальные инструменты разработки.
3. Спроектировать архитектуру приложения, включая клиентскую часть (Android) и взаимодействие с серверными API.

Объектом исследования является процесс разработки мобильных приложений для мониторинга физической активности.

Предметом исследования является разработка мобильного приложения с использованием встроенных датчиков смартфона, клиент-серверного взаимодействия и современных подходов к проектированию пользовательского интерфейса.

Практическая значимость работы заключается в создании бесплатного и точного инструмента для отслеживания физической активности, который может быть использован как индивидуальными пользователями, так и в рамках корпоративных wellness-программ. Приложение также предоставляет дополнительные функции (погода, новости), что повышает его удобство и вовлеченность пользователей.

Структура и объем курсового проекта. Курсовой проект состоит из введения, двух глав, заключения, списка использованных источников (15 наименований) и приложения. Общий объем работы составляет 61 страниц, включая 13 рисунков и 7 листингов кода, демонстрирующих архитектуру

и программную реализацию основных компонентов приложения. Объем основной части курсового проекта составляет 49 страниц.

Во введении обосновывается актуальность темы, формулируются цель и задачи исследования, определяются объект и предмет работы, раскрывается ее практическая значимость, а также описывается структура курсового проекта.

В первой главе рассматриваются теоретические основы разработки мобильных приложений для мониторинга физической активности: представлена классификация мобильных приложений по категориям, проведен сравнительный анализ существующих решений (Google Fit, Samsung Health, MyFitnessPal, Strava, Nike Training Club), выполнен анализ технологий и языков программирования для Android-разработки, описаны основные этапы создания приложения и обоснован выбор платформы и инструментов.

Во второй главе подробно описан процесс проектирования и разработки мобильного приложения FitTrack: представлена архитектура приложения на основе MVVM и Clean Architecture, рассмотрена реализация сервисного слоя и интеграция с внешними API, описаны алгоритмы обработки данных с датчиков и сервисов, детально проанализирована система локального хранения данных на основе Room, освещены вопросы оптимизации производительности, многопоточной обработки и тестирования приложения.

В заключении подведены итоги разработки, отмечены преимущества созданного приложения и обозначены перспективы его дальнейшего развития. Приложение содержит листинги ключевых компонентов программного кода с подробными комментариями.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ МОБИЛЬНЫХ ПРИЛОЖЕНИЙ ДЛЯ МОНИТОРИНГА ФИЗИЧЕСКОЙ АКТИВНОСТИ

1.1. Общая информация о мобильных приложениях и их категориях.

Мобильные приложения представляют собой программное обеспечение, разработанное для работы на смартфонах, планшетах и других мобильных устройствах [4]. Они стали неотъемлемой частью современной жизни, предоставляя пользователям доступ к различным сервисам, развлечениям, инструментам для работы и здоровья. В зависимости от функционального назначения мобильные приложения можно разделить на несколько основных категорий [8]. Социальные сети и коммуникации - приложения, предназначенные для общения, обмена сообщениями и контентом (например, WhatsApp, Instagram, Telegram). Они помогают пользователям оставаться на связи, делиться новостями и взаимодействовать в режиме реального времени [10]. Развлекательные приложения включают в себя стриминговые сервисы (YouTube, Netflix), игры, приложения для чтения книг и прослушивания музыки. Их основная задача - обеспечить пользователям досуг и расслабление [3]. Продуктивность и бизнес - это приложения для организации работы, управления задачами, ведения финансов и документооборота (Google Docs, Trello, Excel). Они помогают оптимизировать рабочие процессы и повышать эффективность [12]. Здоровье и фитнес - категория, к которой относится разрабатываемое приложение FitTrack. Сюда входят трекеры активности, приложения для медитации, подсчета калорий и тренировок (MyFitnessPal, Strava, Nike Training Club). Они помогают пользователям следить за физическим состоянием, мотивируют на ведение здорового образа жизни и предоставляют полезные данные для самоанализа [7]. Новостные и информационные приложения агрегируют актуальные новости, статьи и аналитику (Google News, РБК). Они позволяют пользователям оставаться в курсе событий и получать

проверенную информацию [8]. Погодные приложения предоставляют прогнозы, предупреждения и рекомендации на основе метеорологических данных (AccuWeather, Яндекс.Погода). Они особенно важны для планирования повседневных дел и путешествий [10]. Утилиты и инструменты - это вспомогательные приложения, такие как калькуляторы, сканеры QR-кодов, переводчики и файловые менеджеры. Они упрощают выполнение повседневных задач [4]. Каждая категория приложений решает определенные потребности пользователей, и их разработка требует учета специфики целевой аудитории, технических возможностей устройств и интеграции с другими сервисами [3]. В последние годы наблюдается рост спроса на приложения, связанные с здоровьем и фитнесом, что обусловлено повышением интереса к саморазвитию и мониторингу состояния организма [9]. Это делает данную категорию особенно перспективной для разработки инновационных решений.

1.2. Сравнительный анализ существующих решений.

Современный рынок мобильных приложений для фитнеса представляет собой динамично развивающуюся экосистему, где ежегодно появляются сотни новых решений [7]. Однако при таком многообразии выбора пользователи часто сталкиваются с проблемой поиска действительно качественного и функционального продукта, который бы удовлетворял все их потребности. Проведем детальный анализ популярных и востребованных приложений данного сегмента, чтобы выявить их конкурентные преимущества, функциональные ограничения и потенциальные направления для улучшения.



Рис. 1.1 Google Fit

Google Fit [16] представляет собой кроссплатформенное решение, разработанное компанией Google в рамках экосистемы Android. Это приложение ориентировано на базовый мониторинг физической активности и включает такие функции как точный подсчет шагов, измерение пройденного расстояния, мониторинг сердечного ритма (при условии подключения совместимых носимых устройств), а также учет различных видов тренировок. Среди его неоспоримых преимуществ стоит отметить глубокую интеграцию с операционной системой Android, что обеспечивает стабильную работу и минимальное потребление ресурсов устройства. Интерфейс приложения выполнен в соответствии с принципами Material Design, что делает его интуитивно понятным даже для неопытных пользователей. Однако анализ отзывов и экспертных оценок выявляет ряд существенных недостатков: ограниченные возможности аналитики собранных данных, отсутствие продвинутых функций для профессиональных спортсменов, а также слабая система мотивации пользователей для поддержания регулярной активности.

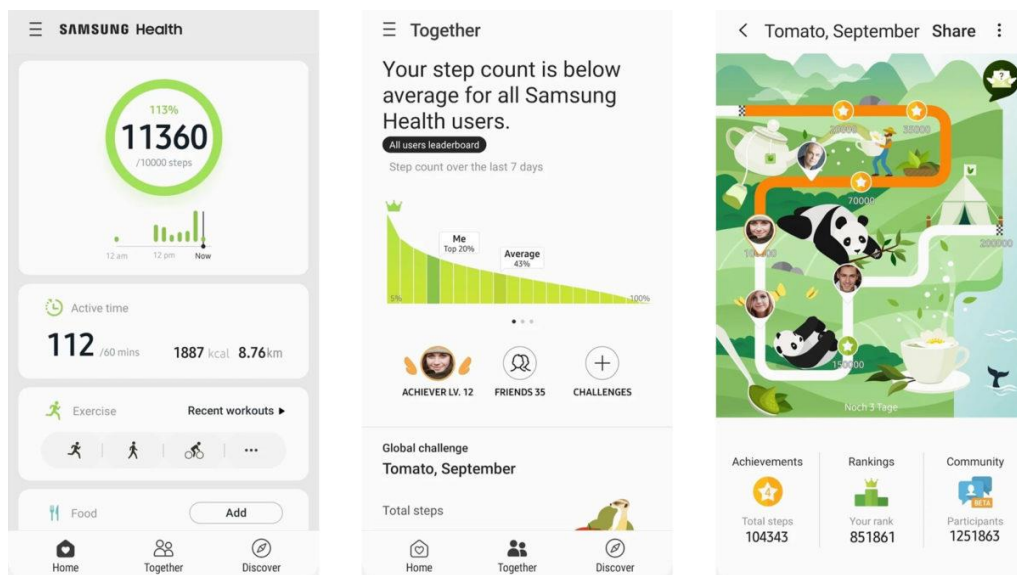


Рис. 1.2. Samsung Health

Samsung Health [17] - это комплексное решение от южнокорейского технологического гиганта Samsung, которое изначально разрабатывалось как эксклюзивное приложение для устройств этого производителя. Со временем оно эволюционировало в многофункциональную платформу для здоровья, предлагающую не только стандартный трекинг физической активности, но и уникальные функции вроде мониторинга качества сна, контроля стрессовых показателей, измерения насыщения крови кислородом и даже анализа состава тела (при использовании совместимых весов Samsung). Технологическая база приложения позволяет достигать исключительной точности измерений благодаря использованию всего спектра датчиков современных смартфонов Samsung. Однако именно эта привязка к экосистеме производителя становится главным ограничением - многие продвинутые функции остаются недоступными для пользователей устройств других брендов. Кроме того, интерфейс приложения перегружен многочисленными опциями, что может вызывать сложности у неподготовленных пользователей.

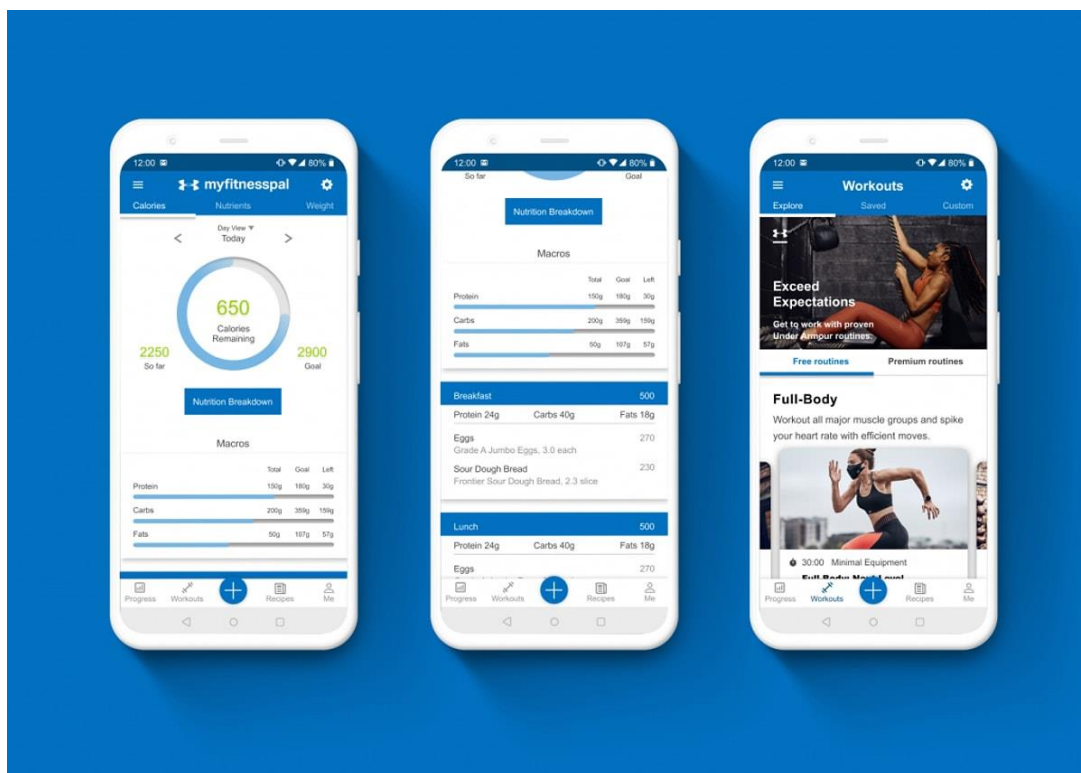


Рис. 1.3. MyFitnessPal

MyFitnessPal [18] по праву считается одним из лидеров в сегменте приложений для контроля питания и здорового образа жизни. Его ключевой особенностью является огромная база данных пищевых продуктов (свыше 11 миллионов наименований) с детальной информацией о питательной ценности. Приложение интегрирует функции трекинга физической активности, позволяя пользователям видеть полную картину баланса между потребленными и потраченными калориями. Система сканирования штрих-кодов продуктов и возможность создания собственных рецептов делают процесс учета питания максимально удобным. Однако бизнес-модель разработчиков построена на премиум-подписке, что означает ограничение доступа ко многим аналитическим функциям и персонализированным рекомендациям в бесплатной версии. Это существенно снижает ценность продукта для пользователей, не готовых к дополнительным расходам.

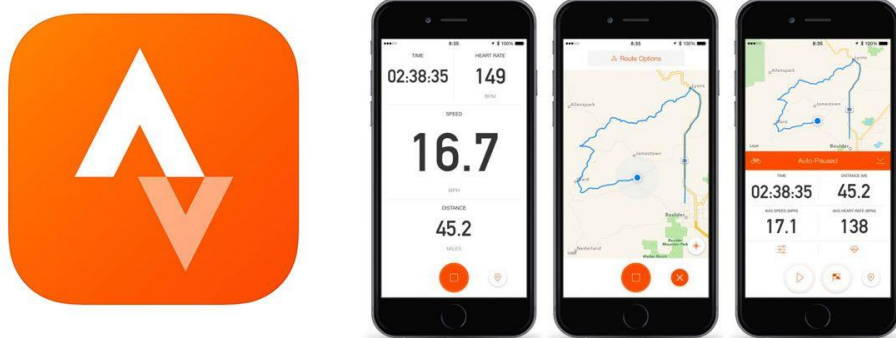


Рис. 1.4. Strava

Strava [19] занимает особую нишу среди фитнес-приложений, позиционируясь как социальная сеть для спортсменов. Основной акцент сделан на точном трекинге беговых и велосипедных маршрутов с детальным анализом таких параметров как скорость, темп, высота над уровнем моря и даже мощность педалирования (для велосипедистов). Уникальной особенностью является система сегментов - определенных участков маршрутов, где пользователи могут соревноваться друг с другом. Социальные функции включают возможность делиться своими достижениями, оставлять комментарии и создавать тематические клубы по интересам. Однако такая узкая специализация одновременно является и главным ограничением - приложение практически бесполезно для тех, кто занимается другими видами активности или просто хочет контролировать свою ежедневную подвижность.

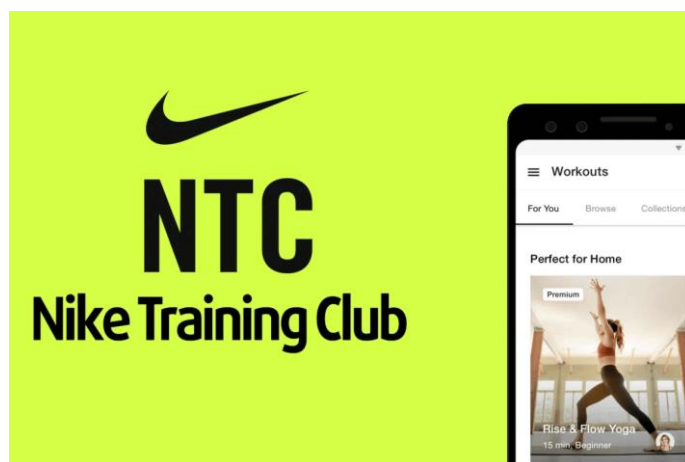


Рис. 1.5. Nike Training Club

Nike Training Club [20] представляет собой полноценную цифровую платформу для фитнеса, предлагающую пользователям доступ к профессиональным тренировочным программам различной направленности и сложности. База данных включает сотни видеоуроков с детальными инструкциями, которые охватывают все основные виды тренировок: от йоги и пилатеса до высокоинтенсивных интервальных занятий. Система персонализации позволяет создавать индивидуальные планы тренировок с учетом уровня подготовки, доступного оборудования и поставленных целей. Особого внимания заслуживает мотивационная система приложения, включающая элементы геймификации и возможность участия в челленджах. Однако, как и в случае с MyFitnessPal, большая часть контента доступна только по подписке, что существенно ограничивает функциональность бесплатной версии.

Проведенный анализ позволяет сделать несколько важных выводов о текущем состоянии рынка фитнес-приложений. Во-первых, большинство решений демонстрируют явную тенденцию к специализации, фокусируясь либо на определенных видах активности (как Strava), либо на отдельных аспектах здорового образа жизни (контроль питания в MyFitnessPal). Во-вторых, бизнес-модели многих популярных приложений построены на ограничении функциональности бесплатных версий, что вынуждает пользователей приобретать подписки. В-третьих, лишь немногие разработчики уделяют внимание интеграции дополнительных сервисов, которые могли бы повысить удобство и практическую ценность приложений (например, данных о погодных условиях или актуальной информации о здоровье).

Именно эти наблюдения подтверждают актуальность разработки комплексного решения, каким является приложение FitTrack. Его конкурентными преимуществами станут: использование всего потенциала встроенных датчиков современных смартфонов для максимально точного мониторинга активности, уникальная комбинация фитнес-трекинга с

актуальной информацией о погодных условиях и новостями в сфере здоровья, система интеллектуальных рекомендаций, адаптирующихся не только к уровню активности пользователя, но и к внешним условиям. Особое внимание уделяется принципиально бесплатной модели распространения без ограничения функциональности, что делает приложение доступным для самой широкой аудитории. Интуитивно понятный интерфейс, построенный по принципам юзер-центричного дизайна, и плавные анимации переходов обеспечат положительный пользовательский опыт, способствуя формированию устойчивой привычки к регулярной физической активности.

1.3. Анализ технологий и языков программирования.

Разработка современного мобильного приложения, особенно в такой требовательной сфере как фитнес-трекинг, требует комплексного подхода к выбору технологического стека [14]. Оптимальное сочетание языков программирования, фреймворков и архитектурных решений должно обеспечивать не только стабильную работу приложения в текущий момент, но и его масштабируемость, безопасность и простоту дальнейшей поддержки. В контексте создания фитнес-приложения FitTrack для платформы Android проведем детальный анализ всех ключевых технологических аспектов разработки.

Языки программирования.

Основным языком разработки для приложения выбран Kotlin - современный статически типизированный язык, который с 2019 года официально рекомендован Google как приоритетный для Android-разработки [2]. По сравнению с традиционной Java, Kotlin предлагает ряд существенных преимуществ: Более лаконичный и выразительный синтаксис, позволяющий сократить объем кода на 30-40%

1. Полная совместимость с Java и возможность использования всей существующей экосистемы библиотек
2. Встроенные механизмы защиты от null pointer exceptions

3. Поддержка как объектно-ориентированного, так и функционального подхода

4. Корутины для эффективной работы с асинхронными операциями

5. Отличная интеграция с Android Studio и всеми инструментами разработки

Для отдельных модулей, требующих особой производительности (например, обработки данных с датчиков), используется сочетание Kotlin с нативным кодом на C++ через Android NDK (Native Development Kit).

Архитектурные решения:

Архитектура приложения построена на комбинации двух современных подходов:

1. Clean Architecture - обеспечивает четкое разделение на уровни (presentation, domain, data) и независимость бизнес-логики от деталей реализации

2. MVVM (Model-View-ViewModel) - паттерн представления данных, идеально подходящий для Android-приложений

Такое сочетание позволяет достичь:

- Полного разделения ответственности между компонентами
- Упрощенного модульного и интеграционного тестирования
- Гибкости при внесении изменений и добавлении нового функционала

- Устойчивости к изменениям внешних API и требований

- Простоты поддержки и рефакторинга

Для реализации архитектуры используются компоненты Android Jetpack:

- ViewModel для хранения и управления UI-данными
- LiveData для реактивного программирования
- Room для работы с базой данных
- Navigation Component для управления переходами между экранами

Пользовательский интерфейс:

Для создания UI применяется современный декларативный фреймворк Jetpack Compose, который предлагает:

1. Декларативный подход к построению интерфейсов
2. Полную совместимость с существующими View-компонентами
3. Встроенную поддержку Material Design
4. Упрощенную систему анимаций и переходов
5. Повышенную производительность за счет интеллектуальной

перерисовки только изменяющихся элементов

Работа с данными:

Для различных аспектов работы с данными используются специализированные библиотеки:

1. Хранение данных:
 - Room (абстракция над SQLite) - для структурированных данных
 - DataStore (Preferences или Proto) - для простых ключ-значений
 - WorkManager - для управления фоновыми задачами
2. Сетевые взаимодействия:
 - Retrofit - типобезопасный HTTP-клиент
 - OkHttp - для низкоуровневых сетевых операций
 - Gson/Moshi - для сериализации/десериализации JSON
3. Кэширование:
 - Coil - для загрузки и кэширования изображений
 - Room с поддержкой RxJava/Coroutines - для оффлайн-доступа

Работа с датчиками:

Для точного трекинга активности используются:

1. Android Sensor Framework:
 - Акселерометр для подсчета шагов
 - Гироскоп для определения типа активности
 - Датчик освещенности для адаптации интерфейса

2. Activity Recognition API - для автоматического определения вида активности (ходьба, бег, велосипед)

3. Location Services:

- Fused Location Provider для точного позиционирования
- Геозоны для создания маршрутов

Тестирование и качество кода:

Для обеспечения надежности реализована многоуровневая система тестирования:

1. Модульные тесты (JUnit 5, MockK)
2. Интеграционные тесты (Espresso, UI Automator)
3. End-to-end тесты (Firebase Test Lab)
4. Статический анализ (Detekt, ktlint)
5. Непрерывная интеграция (GitHub Actions)

Выбор именно этого технологического стека обусловлен следующими ключевыми факторами:

1. Поддержка со стороны Google и активное сообщество
2. Хорошая документация и обучающие материалы
3. Совместимость с 95% Android-устройств
4. Оптимальный баланс между производительностью и скоростью разработки
5. Возможности для будущего масштабирования
6. Безопасность и стабильность работы

Такой комплексный подход позволяет создавать высокопроизводительное, надежное и удобное в поддержке приложение, которое эффективно работает даже на устройствах с ограниченными ресурсами, что особенно важно для нашей целевой аудитории.

1.4. Основные этапы разработки.

Разработка мобильного приложения FitTrack проходила в несколько последовательных этапов, каждый из которых имел свои особенности и

задачи [8]. Процесс создания приложения начался с детального анализа требований и оценки существующих решений на рынке. Было проведено исследование целевой аудитории, включающее анкетирование потенциальных пользователей для выявления их предпочтений и ожиданий от приложения для отслеживания физической активности [3]. Полученные данные позволили сформировать основной набор функциональных требований, определить приоритетные задачи и обозначить ключевые особенности будущего продукта. На этом этапе был сделан вывод о необходимости интеграции нескольких направлений: отслеживание шагов, мониторинг погоды и предоставление релевантных новостей о здоровье и фитнесе [7]. После завершения аналитического этапа было выполнено проектирование архитектуры приложения. Выбор был сделан в пользу архитектурного паттерна MVVM (Model-View-ViewModel), который обеспечивает четкое разделение ответственности между компонентами и позволяет эффективно организовать взаимодействие между слоями приложения [1]. Данная архитектура особенно хорошо подходит для мобильных приложений с множеством экранов и сложным жизненным циклом компонентов. Для локального хранения данных была спроектирована схема базы данных на основе SQLite с использованием библиотеки Room, позволяющая эффективно кэшировать и обрабатывать информацию [6]. Важным решением стало разделение приложения на модули, каждый из которых отвечал за отдельный функциональный блок, что упростило разработку и последующую поддержку [14]. Следующим этапом стало создание прототипов пользовательского интерфейса. Сначала были разработаны низкодетализированные макеты экранов с помощью инструмента прототипирования, что позволило быстро визуализировать структуру приложения и основные элементы навигации [3]. После согласования базовой концепции были созданы высокодетализированные макеты, учитывающие принципы Material Design и особенности пользовательского опыта на мобильных устройствах. Особое

внимание уделялось интуитивности интерфейса, легкости восприятия информации и доступности для пользователей с различными потребностями [3]. На данном этапе также были определены цветовые схемы, типографика и иконография приложения. Непосредственная разработка приложения началась с создания базовой структуры проекта и настройки среды разработки. Был выбран язык программирования Kotlin как современный, выразительный и безопасный инструмент для разработки под Android [2]. Реализация проходила итеративно, начиная с базовых компонентов и постепенно добавляя более сложную функциональность [8]. Сначала была разработана навигационная структура приложения с использованием фрагментов и нижней навигационной панели, затем последовательно реализованы основные экраны: главный экран с отображением статистики шагов, экран детальной статистики, экран погоды, новостной раздел и настройки приложения [13]. Особого внимания заслуживает процесс разработки модуля шагомера, который является ключевым компонентом приложения. Для подсчета шагов был использован сенсор акселерометра в комбинации с Activity Recognition API, что позволило достаточно точно определять физическую активность пользователя [9]. Реализация включала создание фоновой службы, который продолжает работать даже при свернутом приложении, сохраняя данные о шагах в локальной базе данных. Для обеспечения бесперебойной работы службы при различных сценариях использования устройства были применены механизмы Foreground Service с уведомлением, что соответствует современным требованиям Android к фоновым процессам [11]. Интеграция погодного модуля потребовала взаимодействия с внешним API OpenWeatherMap. Реализация включала создание сервисного слоя для работы с сетевыми запросами с использованием библиотеки Retrofit, обработку полученных данных и их отображение в пользовательском интерфейсе [10]. Особое внимание было уделено обработке ошибок и реализации стратегии кэширования данных для оптимизации сетевого трафика и обеспечения

работы приложения в офлайн-режиме. Дополнительно была разработана логика для формирования рекомендаций по физической активности на основе текущих погодных условий [7]. Разработка новостного модуля представляла собой комплексную задачу, включающую не только интеграцию с News API для получения данных, но и реализацию многоуровневой системы фильтрации для отбора наиболее релевантных новостей о здоровье и фитнесе [10]. На первом этапе был создан базовый алгоритм поиска, по ключевым словам, который затем был дополнен более сложной логикой анализа контента для определения его релевантности тематике приложения. Значительное внимание было уделено оптимизации отображения новостей, включая корректную обработку HTML-контента и очистку текста от нежелательных символов и форматирования [13]. Параллельно с основной разработкой велась работа над улучшением пользовательского опыта. Были реализованы различные анимации для переходов между экранами, обновления данных и взаимодействия с элементами интерфейса [3]. Особое внимание уделялось отзывчивости приложения при выполнении ресурсоемких операций, для чего использовались корутины Kotlin, позволяющие эффективно управлять асинхронными задачами без блокировки основного потока [15]. Также была реализована поддержка различных конфигураций экрана и адаптация интерфейса для различных размеров дисплеев [4]. Важным этапом разработки стала реализация механизма сохранения настроек пользователя, включая предпочтения по теме оформления, целевому количеству шагов и выбранному городу для отображения погоды. Для этого был использован механизм SharedPreferences, позволяющий эффективно хранить примитивные типы данных [4]. Реализация смены темы оформления включала как переключение между светлой и темной темами, так и корректное применение цветовых схем ко всем компонентам приложения с учетом рекомендаций Material Design [3]. Тестирование приложения проводилось на различных этапах разработки и включало несколько уровней проверки [8].

Модульное тестирование с использованием JUnit позволило проверить корректность работы отдельных компонентов, таких как сервисы, репозитории и утилитные классы. Интеграционное тестирование помогло выявить проблемы взаимодействия между различными модулями приложения. UI-тестирование с использованием Espresso позволило автоматизировать проверку пользовательского интерфейса и сценариев взаимодействия [13]. Дополнительно проводилось мануальное тестирование на различных устройствах для выявления проблем совместимости и оптимизации производительности. Особое внимание было уделено оптимизации производительности приложения. Были проведены профилирование и анализ использования памяти, что позволило выявить и устранить узкие места [7]. Оптимизация включала эффективное кэширование данных, минимизацию числа сетевых запросов, использование ленивой загрузки ресурсов и применение паттернов эффективного рендеринга пользовательского интерфейса [15]. Для обеспечения плавности работы приложения были реализованы механизмы пагинации при загрузке больших объемов данных, таких как новостные статьи [10]. Финальный этап разработки включал подготовку приложения к релизу: оптимизацию размера APK-файла, конфигурацию ProGuard для обфускации кода и удаления неиспользуемых ресурсов, создание подписанной версии приложения [12]. Был подготовлен набор маркетинговых материалов, включая скриншоты приложения и описание его функциональности для публикации в Google Play Store. Дополнительно была настроена система аналитики Firebase для отслеживания пользовательской активности и основных метрик использования приложения, что позволит в дальнейшем анализировать поведение пользователей и оптимизировать функциональность [11]. Таким образом, разработка приложения FitTrack прошла все необходимые этапы от концепции до готового продукта, с учетом современных подходов к созданию мобильных приложений [8], обеспечивая высокое качество кода,

удобство использования и соответствие потребностям целевой аудитории. Итоговое решение представляет собой комплексный инструмент для мониторинга физической активности с дополнительными функциями, повышающими его ценность для пользователей [7].

1.5. Выбор платформы и инструментов.

Разработка мобильного приложения FitTrack проходила в несколько последовательных этапов, каждый из которых имел свои особенности и задачи [8]. Процесс создания приложения начался с детального анализа требований и оценки существующих решений на рынке. Было проведено исследование целевой аудитории, включающее анкетирование потенциальных пользователей для выявления их предпочтений и ожиданий от приложения для отслеживания физической активности [3]. Полученные данные позволили сформировать основной набор функциональных требований, определить приоритетные задачи и обозначить ключевые особенности будущего продукта. На этом этапе был сделан вывод о необходимости интеграции нескольких направлений: отслеживание шагов, мониторинг погоды и предоставление релевантных новостей о здоровье и фитнесе [7].

В качестве основного языка программирования был выбран Kotlin, который с 2019 года рекомендован Google как предпочтительный язык для разработки приложений под Android. Kotlin обладает рядом преимуществ по сравнению с традиционно используемой Java: он более краткий и выразительный, что позволяет писать более читаемый и поддерживаемый код; обеспечивает высокий уровень безопасности типов, предотвращая распространенные ошибки `NullPointerException`; полностью совместим с существующим Java-кодом, что облегчает интеграцию с legacy-компонентами и библиотеками. Кроме того, Kotlin предоставляет ряд современных языковых возможностей, таких как расширения функций, делегированные свойства, лямбда-выражения и корутины, которые значительно упрощают разработку

асинхронных операций, необходимых для работы с сетевыми запросами и базами данных.

Основной средой разработки был выбран Android Studio – официальная интегрированная среда разработки (IDE) для Android, созданная на базе IntelliJ IDEA от JetBrains. Android Studio предоставляет все необходимые инструменты для эффективной разработки, включая мощный редактор кода с интеллектуальным автодополнением, интегрированную систему сборки Gradle, визуальный редактор макетов с предпросмотром в реальном времени, встроенные инструменты профилирования и отладки, а также множество шаблонов для быстрого создания типовых компонентов приложения. Версионный контроль осуществлялся с использованием Git, что обеспечивало надежное отслеживание изменений в коде и возможность командной работы над проектом. Для структурирования архитектуры приложения был выбран паттерн MVVM (Model-View-ViewModel) в сочетании с компонентами Android Architecture Components из библиотеки Jetpack. Такой подход позволяет эффективно разделить ответственность между различными частями приложения, обеспечивая лучшую тестируемость, поддерживаемость и масштабируемость кода. В частности, использовались следующие компоненты: ViewModel для сохранения состояния UI независимо от жизненного цикла активностей и фрагментов; LiveData для реактивного обновления пользовательского интерфейса при изменении данных; Room для абстракции над SQLite и удобной работы с локальными базами данных; Navigation для управления переходами между экранами приложения. Для реализации пользовательского интерфейса были использованы современные компоненты из библиотеки Material Components for Android, которая предоставляет набор готовых элементов UI, соответствующих принципам Material Design от Google. Это позволило создать визуально привлекательный, интуитивно понятный и соответствующий современным трендам интерфейс приложения. Для отображения динамических списков, таких как список новостей или история активности, применялся компонент RecyclerView,

обеспечивающий эффективное отображение больших наборов данных с минимальным потреблением ресурсов. Для организации экранов использовались преимущественно `ConstraintLayout` и `MotionLayout`, позволяющие создавать гибкие и отзывчивые макеты, адаптирующиеся к различным размерам экранов и ориентациям устройства. Для работы с сетевыми запросами была выбрана библиотека `Retrofit` в сочетании с `OkHttp` и `Gson`. `Retrofit` предоставляет удобный и типобезопасный способ определения API-клиентов с использованием аннотаций, что значительно упрощает интеграцию с внешними сервисами. `OkHttp` используется в качестве эффективного HTTP-клиента с поддержкой кэширования, перехватчиков запросов и других полезных функций. `Gson` применяется для сериализации и десериализации JSON-объектов, обеспечивая преобразование между сетевыми данными и объектами Kotlin. Такой стек технологий позволил эффективно реализовать взаимодействие с погодным API `OpenWeatherMap` и новостным API `NewsAPI`.

Для загрузки и кэширования изображений в новостном разделе применялась библиотека `Picasso`, которая отличается простотой использования, автоматическим кэшированием и эффективным управлением памятью. `Picasso` позволяет асинхронно загружать изображения из сети, применять к ним различные трансформации и плавно отображать в пользовательском интерфейсе, не вызывая блокировки основного потока и сохраняя отзывчивость приложения.

Для работы с многопоточностью и асинхронными операциями применялись корутины Kotlin, которые предоставляют элегантный и эффективный способ написания неблокирующего кода без использования сложных конструкций обратных вызовов (callbacks) или цепочек RxJava. Корутины особенно полезны при выполнении длительных операций, таких как запросы к сети или базе данных, и позволяют писать асинхронный код в последовательном стиле, что значительно улучшает его читаемость и поддерживаемость.

Для хранения пользовательских настроек и конфигурации приложения использовался механизм `SharedPreferences`, предоставляющий простой способ сохранения примитивных типов данных в виде пар ключ-значение. Этот подход идеально подходит для хранения таких параметров, как выбранный город для погоды, целевое количество шагов, предпочтения по теме оформления и другие настройки пользователя.

Для тестирования различных аспектов приложения применялся комплексный набор инструментов. Модульные тесты создавались с использованием `JUnit` и `Mockito`, что позволяло изолированно проверять логику отдельных компонентов. Для интеграционного тестирования использовались инструменты из `AndroidX Test`, включая библиотеку `Truth` для более выразительных утверждений. UI-тестирование проводилось с помощью `Espresso`, который предоставляет мощный API для автоматизации пользовательских действий и проверки состояния пользовательского интерфейса. Для отслеживания ошибок и поведения приложения в реальных условиях была интегрирована библиотека `Firebase Crashlytics`, которая автоматически собирает информацию о сбоях и ошибках, позволяя быстро выявлять и устранять проблемы. Дополнительно использовался `Firebase Analytics` для сбора анонимных данных об использовании различных функций приложения, что в перспективе позволяет оптимизировать пользовательский опыт на основе реальных паттернов использования. Для реализации фоновых процессов, таких как подсчет шагов даже при свернутом приложении, использовались сервисы `Android`, включая `Foreground Service` с постоянным уведомлением, что соответствует современным требованиям `Android` к фоновым процессам. Доступ к датчикам устройства, таким как акселерометр и шагомер, осуществлялся через `SensorManager API`, а для более точного распознавания активности применялся `ActivityRecognition API` из пакета `Google Play Services`.

Сборка и управление зависимостями проекта осуществлялись с помощью системы сборки `Gradle`, которая обеспечивает гибкое

конфигурирование процесса сборки, управление различными вариантами сборки (debug, release) и эффективное разрешение зависимостей между модулями и внешними библиотеками. Для улучшения процесса разработки использовались различные плагины Gradle, такие как kotlin-kapt для процессинга аннотаций, kotlin-parcelize для автоматического генерирования кода Parcelable и navigation-safe-args для типобезопасной навигации между фрагментами.

При подготовке релизной версии приложения применялись инструменты для оптимизации размера APK-файла, включая R8 для минификации и обфускации кода, а также Android App Bundle для генерации оптимизированных версий приложения для различных конфигураций устройств. Это позволило уменьшить размер устанавливаемого пакета и сократить время загрузки приложения пользователями.

Таким образом, выбор платформы Android и сопутствующих инструментов разработки был оптимальным для реализации всех требуемых функций приложения FitTrack, обеспечивая необходимую гибкость, производительность и потенциал для дальнейшего масштабирования. Использование современного стека технологий, включая Kotlin, MVVM, Android Jetpack и Material Design, позволило создать высококачественное приложение, соответствующее текущим стандартам разработки для платформы Android и предоставляющее пользователям эффективный инструмент для отслеживания физической активности, получения релевантной информации о погоде и новостей о здоровье и фитнесе.

Выводы по первой главе.

В ходе анализа предметной области установлена высокая востребованность приложений для здоровья и фитнеса, что подтверждает актуальность разработки FitTrack. Сравнительный анализ существующих решений выявил их основные недостатки: узкую специализацию, ограничения в бесплатных версиях и слабую интеграцию с дополнительными сервисами.

Конкурентными преимуществами FitTrack определены комплексность функционала и бесплатная модель распространения без ограничений.

Обоснован выбор Kotlin и архитектурного подхода MVVM с Clean Architecture, обеспечивающих оптимальное сочетание производительности, поддерживаемости и тестируемости кода. Описание этапов разработки продемонстрировало системный подход к созданию продукта: от прототипирования до реализации ключевых модулей (шагомер, погодный и новостной компоненты) и их интеграции в единую экосистему. Использование платформы Android с современным стек технологий (Jetpack, Material Components, Retrofit, Room) создает надежную основу для реализации конкурентоспособного приложения, предоставляющего пользователям комплексный инструмент для поддержания здорового образа жизни.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ FITTRACK

2.1. Проектирование архитектуры приложения

Для создания эффективной архитектуры приложения FitTrack была выбрана архитектура MVVM (Model-View-ViewModel) в сочетании с принципами Clean Architecture. Данное решение позволило обеспечить четкое разделение ответственности между компонентами, повысить модульность, тестируемость кода и общую гибкость системы, что особенно важно для приложения с множеством функций.

На рисунке 2.1 Демонстрируется я главный интерфейс приложения FitTrack с основными метриками активности пользователя: количеством шагов, пройденным расстоянием, сожженными калориями и прогрессом в достижении дневной цели. Интерфейс выполнен в соответствии с принципами Material Design и демонстрирует реактивность компонентов UI.

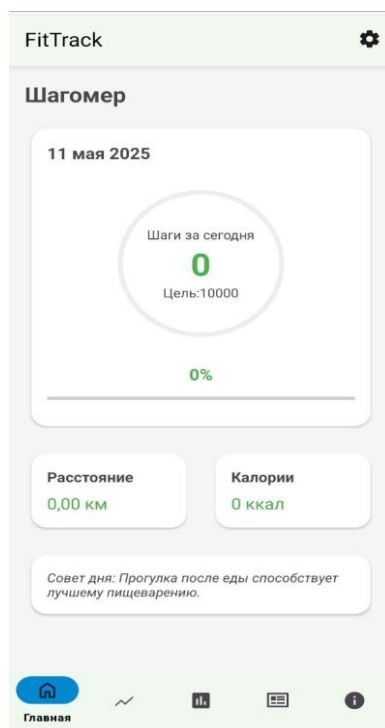


Рис. 2.1. Главный экран приложения

При проектировании архитектуры приложения FitTrack основное внимание уделялось созданию модульной, расширяемой и легко поддерживаемой структуры. Выбор MVVM был обусловлен несколькими ключевыми факторами:

1. Разделение ответственности: MVVM четко разграничивает представление (View), бизнес-логику (ViewModel) и данные (Model), что упрощает поддержку и развитие приложения. Такое разделение позволяет нескольким разработчикам одновременно работать над различными частями приложения, не вызывая конфликтов и не нарушая целостность системы.

2. Поддержка со стороны Android Jetpack: Architecture Components от Google (LiveData, ViewModel, Room) идеально интегрируются с MVVM, предоставляя готовые механизмы для реализации реактивного UI и управления жизненным циклом компонентов. Эти инструменты обеспечивают автоматическое обновление пользовательского интерфейса при изменении данных и корректную обработку изменений конфигурации устройства (например, поворот экрана).

3. Тестируемость: Изоляция бизнес-логики в ViewModel позволяет легко создавать модульные тесты без необходимости имитировать сложные компоненты Android. Благодаря этому можно протестировать большую часть функциональности приложения без необходимости запуска на реальном устройстве или эмуляторе.

Для дальнейшего улучшения структуры проекта были применены принципы Clean Architecture, разделяющие приложение на три основных слоя:

- Presentation Layer (UI): Включает в себя Activity, Fragment, Adapter и ViewModel. Этот слой отвечает за отображение данных и взаимодействие с пользователем. Фрагменты связаны с соответствующими ViewModel через механизм LiveData и DataBinding, как видно в листинге А.7. Данный уровень обеспечивает реактивность интерфейса и правильную обработку жизненного цикла компонентов Android, что критически важно для обеспечения стабильной работы приложения и предотвращения утечек памяти.

- **Domain Layer (Business Logic):** Содержит Use Cases (или Interactor), которые инкапсулируют бизнес-правила и логику приложения. Этот слой полностью независим от Android SDK, что обеспечивает его высокую тестируемость. В нашем приложении здесь расположены такие компоненты как `CalculateCaloriesUseCase`, `TrackStepsUseCase`, `ProcessWeatherDataUseCase`, которые реализуют основные функциональные возможности системы. Независимость от Android SDK позволяет тестировать эту логику в изолированных JUnit-тестах без использования инструментальных тестов.

- **Data Layer (Repository):** Отвечает за получение и хранение данных из различных источников. Репозитории предоставляют единый интерфейс для доступа к данным, скрывая детали их происхождения (локальная база данных, сетевые API или датчики устройства), как показано в листинге А.6. Этот слой обеспечивает прозрачность для вышележащих уровней -бизнес-логика не знает, откуда приходят данные, что позволяет легко заменять источники данных без изменения остальной части системы.

При реализации было уделено особое внимание принципу инверсии зависимостей (DIP), согласно которому модули высокого уровня не зависят от модулей низкого уровня, а оба зависят от абстракций. Для этого интерфейсы репозитория определены в Domain Layer, а их конкретные реализации - в Data Layer. Это позволяет легко заменять одну реализацию репозитория на другую, например, для целей тестирования, без изменения бизнес-логики. Для обеспечения реактивности и асинхронности в приложении был реализован поток данных с использованием корутин Kotlin и Flow. Koroutines позволяют писать асинхронный код в последовательном стиле, значительно упрощая его понимание и поддержку. Flow используется для передачи потоков данных между слоями приложения, обеспечивая реактивное обновление пользовательского интерфейса при изменении данных в реальном времени. Организация пакетов в проекте была спроектирована по функциональному

принципу, что облегчает навигацию в коде и позволяет быстро находить нужные компоненты.

На рисунке 2.2 отображается боковое меню навигации приложения с доступом к основным функциональным разделам: статистике активности, тренировкам, профилю пользователя, новостям о здоровье и настройкам. Наглядно демонстрирует организацию функциональности в соответствии с паттерном MVVM.

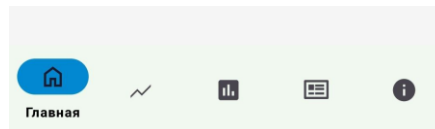


Рис. 2.2. Экран навигации

Основные пакеты включают:

- `com.example.fittrack.ui` -компоненты пользовательского интерфейса, включая активности, фрагменты и адаптеры
- `com.example.fittrack.viewmodel` -компоненты `ViewModel` для управления данными UI
- `com.example.fittrack.domain` -бизнес-логика и модели предметной области
- `com.example.fittrack.data` -источники данных и их репозитории
- `com.example.fittrack.service` -сервисы для фоновой работы
- `com.example.fittrack.util` -вспомогательные классы и утилиты

Такая архитектура обеспечивает множество преимуществ, включая легкость в поддержке и расширении, высокую тестируемость и надежность, а также соответствие современным практикам разработки Android-приложений. Она позволяет добавлять новые функции с минимальными изменениями существующего кода и обеспечивает ясное разделение ответственности между различными частями системы.

2.2. Реализация сервисного слоя и интеграция с API

В рамках разработки приложения была создана комплексная система для взаимодействия как с внешними, так и с системными API Android. Данный компонент играет критическую роль в обеспечении функциональности, связанной с получением погодных данных, новостей о здоровье и фитнесе, а также во взаимодействии с системными сервисами устройства. Для работы с внешними API был создан сервисный слой на основе библиотек Retrofit и OkHttp, как показано в листинге A.5. Retrofit используется для декларативного описания эндпоинтов API и автоматической сериализации/десериализации данных, а OkHttp обеспечивает эффективную работу с HTTP-запросами на низком уровне.

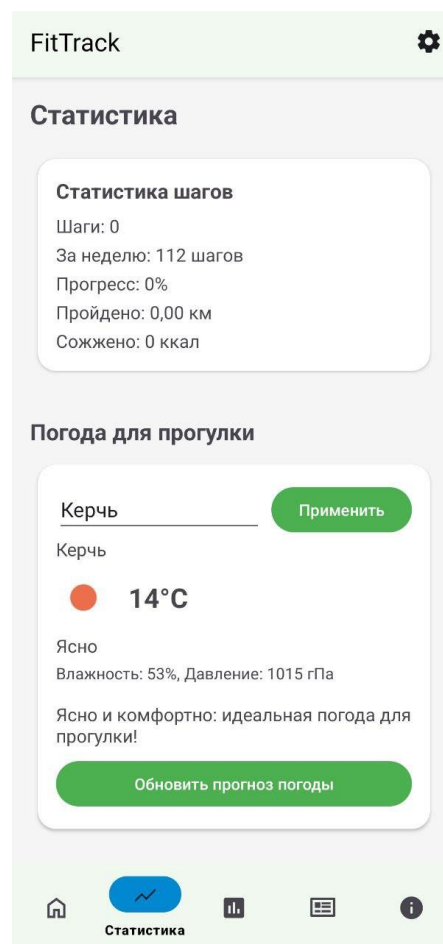


Рис. 2.3. Экран с погодными данными

Были разработаны интерфейсы для взаимодействия с OpenWeatherMap API (для получения погодных данных) и News API (рисунок 2.3 и 2.4).

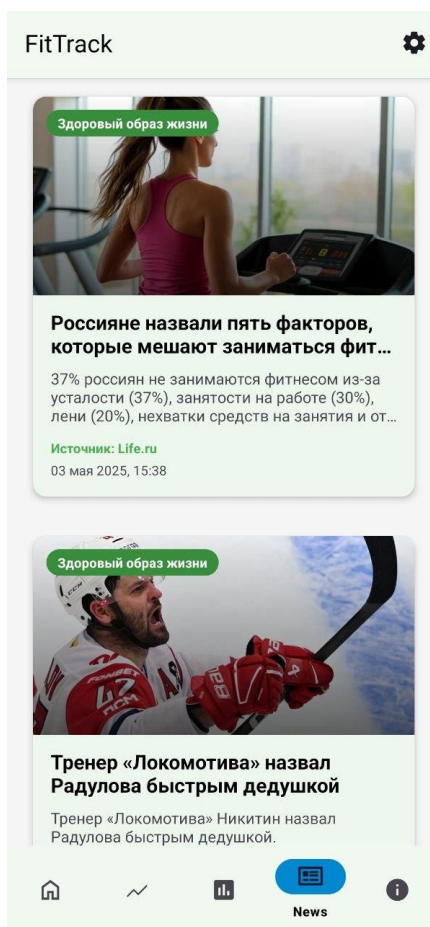


Рис 2.4. Экран новостей о здоровье и фитнесе

В каждом интерфейсе определены методы, соответствующие различным эндпоинтам API, с указанием параметров запросов и типов, возвращаемых данных. Для обеспечения стабильной работы приложения даже в условиях отсутствия сети была реализована комплексная система кэширования данных. Кэширование реализовано на двух уровнях:

1. Уровень HTTP-запросов с использованием механизмов OkHttp, который позволяет кэшировать ответы сервера на основе заголовков Cache-Control
2. Уровень локальной базы данных Room, куда сохраняются полученные данные для последующего использования в оффлайн-режиме

Особое внимание было уделено обработке ошибок и исключительных ситуаций при взаимодействии с API. Для этого была разработана система обертывания ответов в объект Result, который может содержать либо успешный результат, либо информацию об ошибке. Такой подход обеспечивает единообразную обработку различных типов ошибок (проблемы с сетью, ошибки сервера, неверные ответы) и предоставляет информативные сообщения пользователю. Была реализована также система повторных попыток для временных ошибок с экспоненциальной задержкой, что повышает надежность работы с нестабильными сетевыми соединениями. Для эффективной работы с сетевыми запросами и обеспечения отзывчивости пользовательского интерфейса были использованы корутины Kotlin. Они позволяют выполнять сетевые операции асинхронно, не блокируя основной поток приложения, а также легко переключаться между различными контекстами выполнения. Все сетевые запросы выполняются в контексте Dispatchers.IO, что обеспечивает эффективное использование ресурсов устройства и предотвращает блокировки UI-потока. Параллельно была выполнена интеграция с ключевыми системными API Android. Эта интеграция включает в себя несколько компонентов:

1. Взаимодействие с Sensor Framework для получения данных с акселерометра и гироскопа. Для этого был создан абстрактный слой над системным SensorManager, который предоставляет удобный интерфейс для регистрации слушателей сенсоров и обработки их данных. Применены алгоритмы фильтрации, включая низкочастотные фильтры и алгоритмы сглаживания для устранения шумов и повышения точности распознавания движений. Это критически важно для точного подсчета шагов в различных условиях использования.

2. Интеграция с Location API для определения местоположения пользователя при записи маршрутов тренировок и получения локализованных погодных данных. Реализованы различные стратегии получения местоположения в зависимости от требуемой точности и доступности

различных провайдеров (GPS, сеть, пассивный провайдер). Реализованы также механизмы оптимизации потребления энергии при использовании геолокации, включая адаптивную частоту обновления местоположения в зависимости от типа тренировки и уровня заряда батареи устройства.

3. Взаимодействие с `AlarmManager` и `NotificationManager` для создания и отображения уведомлений о достижениях, напоминаний о тренировках и отображения текущей статистики активности. Созданы различные каналы уведомлений с различными приоритетами для разных типов сообщений, что соответствует современным требованиям Android к работе с уведомлениями.

4. Интеграция с `ContentProvider` для доступа к системному календарю и контактам (для функции обмена достижениями), что позволяет приложению взаимодействовать с другими системными приложениями.

Для обеспечения безопасности и контроля доступа к API был разработан слой безопасного хранения API-ключей, который исключает их хранение в открытом виде в исходном коде. Вместо этого ключи хранятся в зашифрованном виде с использованием `EncryptedSharedPreferences` и `AndroidKeyStore`, что защищает их от несанкционированного доступа, как показано в листинге A.1. Кроме того, API-ключи не хранятся непосредственно в коде, а предоставляются через `BuildConfig` в процессе сборки, что позволяет иметь разные ключи для различных сред (разработка, тестирование, продакшн). В процессе интеграции с различными API было уделено особое внимание обработке различных версий Android, с учетом изменений в доступности и поведении API в различных версиях операционной системы. Для этого использовались соответствующие проверки версии SDK и условная логика, а также библиотеки совместимости из пакета `AndroidX` для обеспечения единообразного поведения на различных устройствах.

2.3. Разработка модуля обработки данных с датчиков и сервисов

Ключевой особенностью приложения FitTrack является точный подсчет шагов и определение физической активности пользователя, что требует эффективной обработки данных с датчиков устройства.

Рисунок 2.5. отображает работу алгоритма подсчета шагов в реальном времени с детализацией данных по временным отрезкам. Включает анимированное представление прогресса и детальную статистику активности в течение дня.

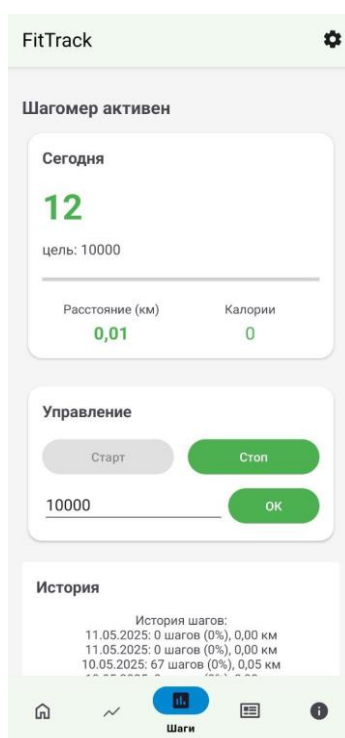


Рис. 2.5. Экран подсчета шагов в реальном времени

Для решения этой задачи был разработан специализированный модуль, обеспечивающий высокую точность и энергоэффективность. Разработан комплексный алгоритм обработки данных акселерометра, состоящий из нескольких ключевых этапов:

1. Предварительная фильтрация входящих данных с применением различных типов фильтров:

- Медианный фильтр для устранения выбросов и случайных шумов

- Фильтр нижних частот для сглаживания сигнала и выделения полезной составляющей

- Адаптивный пороговый фильтр, учитывающий индивидуальные особенности движения

2. Векторизация данных с трех осей акселерометра для получения общей величины ускорения, независимой от ориентации устройства. Это позволяет точно определять шаги независимо от того, где находится смартфон (в кармане, в руке, в сумке).

3. Анализ пиков и впадин в обработанном сигнале для определения характерных паттернов, соответствующих шагам. Для этого применяется алгоритм обнаружения пиков с адаптивным порогом, который учитывает историю движений и автоматически подстраивается под интенсивность активности.

4. Временной анализ для отбрасывания ложных срабатываний на основе типичных временных интервалов между шагами при различных видах активности.

Для повышения точности определения типа активности была реализована интеграция с Activity Recognition API от Google, что позволило различать различные типы движения: неподвижность, ходьба, бег, езда на велосипеде и передвижение в транспорте. Этот механизм обеспечивает контекстную информацию, которая используется для адаптации параметров алгоритма подсчета шагов в зависимости от текущей активности пользователя. Одним из инновационных аспектов разработанного модуля стала система динамической калибровки, которая учитывает индивидуальные особенности походки пользователя. Система работает в двух режимах:

1. Краткосрочная адаптация: анализирует паттерны текущего сеанса активности и быстро подстраивает параметры алгоритма под изменяющиеся условия.

2. Долгосрочное обучение: накапливает статистику о характерных паттернах движения конкретного пользователя на протяжении длительного

времени и формирует персонализированную модель для более точного определения шагов.

Для расчета пройденного расстояния и расхода калорий были разработаны адаптивные алгоритмы, учитывающие не только количество шагов, но и контекстуальную информацию:

- Расчет расстояния учитывает индивидуальную длину шага, которая определяется на основе роста пользователя и корректируется с использованием данных GPS при наличии таковых.
- Расчет калорий основан на метаболических эквивалентах (MET) для различных типов активности и учитывает персональные данные пользователя (вес, пол, возраст), что обеспечивает более точные результаты по сравнению с универсальными формулами.

Рисунок 2.6. демонстрирует компактное, но информативное уведомление, которое отображается при работе Foreground Service. Уведомление показывает текущее количество шагов, прогресс к дневной цели и предоставляет быстрый доступ к основным функциям приложения без необходимости его открытия.

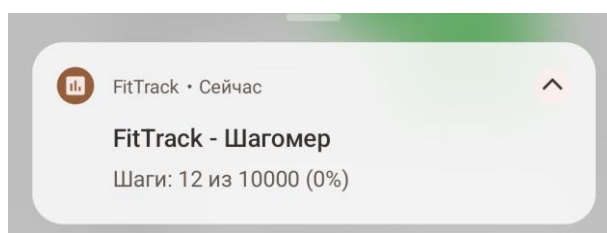


Рис. 2.6. Пример уведомления сервиса подсчета шагов

Для обеспечения непрерывной работы отслеживания активности даже при закрытом приложении была разработана система фоновых сервисов с многоуровневой архитектурой. Основой системы стал Foreground Service с постоянным уведомлением, реализация которого представлена в листинге А.3. Сервис выполняет следующие функции:

1. Регистрирует слушатели для получения данных с сенсоров

2. Обработывает полученные данные в режиме реального времени
3. Сохраняет результаты в локальную базу данных
4. Обновляет уведомление с текущей информацией о шагах

Уведомление не только выполняет требования платформы Android для работы Foreground Service, но и предоставляет пользователю актуальную информацию о количестве шагов и прогрессе в достижении дневной цели без необходимости открывать приложение. Для выполнения периодических задач, таких как синхронизация данных, агрегация статистики и обновление виджетов, был использован WorkManager, который обеспечивает энергоэффективное выполнение отложенных задач с учетом состояния устройства. WorkManager позволяет планировать задачи с различными ограничениями (наличие сети, уровень заряда батареи, подключение к зарядному устройству) и гарантирует их выполнение даже в случае перезагрузки устройства. Для обеспечения автоматического запуска сервисов при перезагрузке устройства реализован механизм на основе BroadcastReceiver, который регистрируется на системное событие BOOT_COMPLETED. При получении данного события приемник проверяет настройки пользователя и, если отслеживание активности включено, запускает соответствующие сервисы. Особое внимание уделялось оптимизации энергопотребления, что является критически важным для приложения, которое должно работать в фоновом режиме длительное время. Для этого реализована адаптивная стратегия опроса датчиков, при которой частота сбора данных автоматически изменяется в зависимости от:

1. Текущей активности пользователя (при неподвижности частота снижается)
2. Уровня заряда батареи (при низком заряде включается энергосберегающий режим)
3. Времени суток (ночью, когда пользователь обычно спит, частота минимальна)

Эта оптимизация позволила существенно снизить энергопотребление при сохранении высокой точности отслеживания, что отражено в листинге А.3 в методах работы с сенсорами. Дополнительно реализован механизм пакетной обработки данных, который позволяет минимизировать количество "пробуждений" процессора для обработки показаний датчиков. Для повышения надежности и устойчивости к сбоям реализована система сохранения и восстановления состояния, которая позволяет корректно продолжить отслеживание после непредвиденных ситуаций, таких как принудительное завершение приложения системой из-за нехватки памяти или перезагрузка устройства.

2.4. Реализация локального хранения данных

Для обеспечения надежного хранения пользовательских данных была спроектирована и реализована комплексная система локального хранения, основанная на современных технологиях и лучших практиках разработки для Android.

Рисунок 2.7. показывает детальную статистику и историю активности пользователя с возможностью анализа данных за различные периоды времени. Демонстрирует эффективную работу с базой данных Room и визуализацию агрегированных данных.



Рис. 2.7. Экран с историей активности

Центральным компонентом этой системы является реляционная база данных SQLite, управляемая через библиотеку Room, которая предоставляет

объектно-реляционный маппинг и защиту от ошибок на этапе компиляции. Схема базы данных включает несколько взаимосвязанных таблиц для хранения различных типов информации. Схема базы данных приложения:

1. StepData -хранит информацию о шагах с детализацией по дням, включая дату, количество шагов, пройденное расстояние, сожженные калории и прогресс относительно дневной цели.
2. ActivitySession -содержит данные о конкретных сеансах активности (тренировках), включая время начала и окончания, тип активности, маршрут и детальную статистику.
3. WeatherData -сохраняет погодные условия для разных местоположений, включая температуру, влажность, атмосферное давление, скорость ветра и описание погоды.
4. NewsItem -хранит новости и статьи, связанные со здоровьем и фитнесом, включая заголовки, краткое описание, полный текст, URL-адреса изображений и метаданные.
5. UserProfile -содержит информацию о пользователе, такую как рост, вес, возраст, пол и предпочтения, которые используются для персонализации расчетов и рекомендаций.

Для каждой из этих сущностей разработан соответствующий DAO-интерфейс (Data Access Object), определяющий методы для взаимодействия с базой данных. Пример такого интерфейса представлен в листинге А.4, где определены методы для работы с данными о шагах. DAO-интерфейсы используют аннотации Room для определения SQL-запросов, что обеспечивает их проверку на этапе компиляции и избавляет от необходимости писать низкоуровневый код для работы с SQLite. При проектировании DAO особое внимание уделялось оптимизации запросов для эффективного получения агрегированных данных и статистики за различные временные периоды. Были реализованы специализированные запросы для получения:

- Ежедневной статистики шагов, расстояния и калорий

- Еженедельных и ежемесячных сводок с агрегированными показателями

- Сравнительных данных для анализа тенденций
- Статистики по разным типам активности
- Рейтингов дней с наилучшими показателями

Оптимизация запросов включала создание соответствующих индексов, использование JOIN для эффективного объединения данных из нескольких таблиц и применение встроенных агрегатных функций SQLite для выполнения расчетов на уровне базы данных, а не в коде приложения. Для обеспечения целостности данных при обновлениях приложения были разработаны механизмы миграции базы данных. Для каждой версии схемы созданы миграционные скрипты, которые автоматически применяются системой Room при обновлении приложения. Миграции реализованы таким образом, чтобы гарантировать сохранность пользовательских данных даже при существенных изменениях структуры базы данных, таких как добавление новых таблиц, изменение схемы существующих таблиц или рефакторинг отношений между ними. На уровне архитектуры приложения взаимодействие с базой данных инкапсулировано в репозиториях, которые предоставляют единый интерфейс для доступа к данным, скрывая детали их происхождения (локальная база данных, сетевые API или датчики устройства). Пример репозитория для работы с данными о шагах представлен в листинге А.6, где реализованы методы для сохранения текущего количества шагов и получения различных видов статистики. Репозитории реализуют шаблон Single Source of Truth, при котором локальная база данных является единственным источником достоверной информации для остальных частей приложения. Данные из внешних источников (сеть, датчики) сначала сохраняются в базу данных, а затем извлекаются из нее для использования в пользовательском интерфейсе. Это обеспечивает консистентность данных и возможность работы приложения в офлайн-режиме. Для эффективного взаимодействия с базой данных

используются корутины Kotlin и Flow. Все операции ввода-вывода выполняются асинхронно в соответствующем диспетчере (Dispatchers.IO), что предотвращает блокировку основного потока и обеспечивает плавность работы пользовательского интерфейса. Flow используется для наблюдения за изменениями в базе данных и автоматического обновления UI при изменении данных. Для хранения пользовательских настроек и других небольших фрагментов данных была реализована специализированная обертка над SharedPreferences, расширяющая стандартную функциональность и добавляющая такие возможности как:

1. Типобезопасный доступ к настройкам с использованием делегатов свойств Kotlin
2. Возможность наблюдать за изменениями конкретных настроек через Flow
3. Автоматическая сериализация и десериализация сложных объектов
4. Поддержка различных типов значений, включая пользовательские классы

Особое внимание уделялось безопасности хранения чувствительной информации, такой как учетные данные пользователя и токены доступа к API. Для этого использовались технологии шифрования на основе EncryptedSharedPreferences и Android Keystore System, которые обеспечивают защиту данных от несанкционированного доступа даже на устройствах с root-правами. Для обеспечения целостности данных при использовании потоков выполнения, в местах, где требуются атомарные операции (например, обновление взаимосвязанных значений), использовались транзакции базы данных. Для долговременного хранения больших блоков данных, таких как логи активности и детальная история использования, была реализована система ротации и архивирования, которая автоматически сжимает и переносит старые данные в отдельные файлы, что позволяет сохранять более долгую историю без чрезмерного увеличения

размера основной базы данных. Реализованный подход к хранению данных обеспечивает надежную основу для функционирования приложения даже при длительном использовании и накоплении больших объемов информации, сохраняя при этом высокую производительность и отзывчивость пользовательского интерфейса.

2.5. Оптимизация производительности и многопоточная обработка.

В процессе разработки FitTrack было проведено детальное профилирование приложения с использованием Android Profiler для выявления узких мест в производительности и потреблении ресурсов.

Результаты профилирования позволили определить области, требующие оптимизации. Для обеспечения отзывчивости интерфейса при выполнении ресурсоемких операций разработана система асинхронной обработки данных на основе корутин Kotlin. Организация потоков данных основана на структурированном подходе с четким разделением ответственности между различными диспетчерами корутин: IO для операций ввода-вывода, Default для интенсивных вычислений и Main для обновления пользовательского интерфейса. Репозитории реализованы с поддержкой Flow для реактивного обновления данных, что позволяет приложению реагировать на изменения в реальном времени. Пример использования Flow и LiveData для обновления пользовательского интерфейса показан в листинге A.7, где реализован ViewModel для экрана статистики. Оптимизированы запросы к базе данных Room путем создания индексов по часто используемым полям и реструктуризации запросов. Внедрено кэширование результатов частых запросов в памяти для снижения нагрузки на базу данных, что особенно важно при работе с большими наборами данных. Реализована система ленивой загрузки ресурсоемких компонентов и техники отложенной инициализации, что значительно сократило время холодного запуска приложения. Значительное внимание уделено оптимизации работы с изображениями и

внешними ресурсами для минимизации потребления памяти. Алгоритмы обработки данных с датчиков были существенно оптимизированы. Внедрены механизмы пакетной обработки и буферизации, позволяющие сократить количество операций записи в базу данных, что снижает общие затраты ресурсов на операции ввода-вывода. Система адаптивной регуляции частоты опроса датчиков автоматически корректирует интенсивность сбора данных, что позволило достичь оптимального баланса между точностью отслеживания и энергоэффективностью, как показано в листинге А.3.

2.6. Тестирование, сборка и развертывание приложения.

Для обеспечения высокого качества кода и надежности приложения была создана комплексная система тестирования. Разработаны модульные тесты на базе JUnit для проверки корректности основных алгоритмов, интеграционные тесты для проверки взаимодействия между слоями приложения, а также инструментальные тесты с использованием Espresso.

Особое внимание уделялось тестированию фоновых сервисов и алгоритмов обработки данных с датчиков. С использованием библиотек Mockito и MockK были созданы имитационные объекты для изоляции тестируемых компонентов от внешних зависимостей, что позволило тщательно протестировать сетевое взаимодействие и обработку данных без реальных запросов к серверу.

Настроен полноценный CI/CD pipeline с использованием GitHub Actions для автоматического запуска комплекса тестов при каждом отправлении изменений в репозиторий.

Реализована система отчетности о покрытии кода тестами, позволяющая контролировать качество тестирования.

Рисунок 2.8 отображает информацию о текущей версии приложения, использованных библиотеках и авторских правах. Этот экран

также представляет результат процесса сборки и демонстрирует использование автоматической генерации версии.

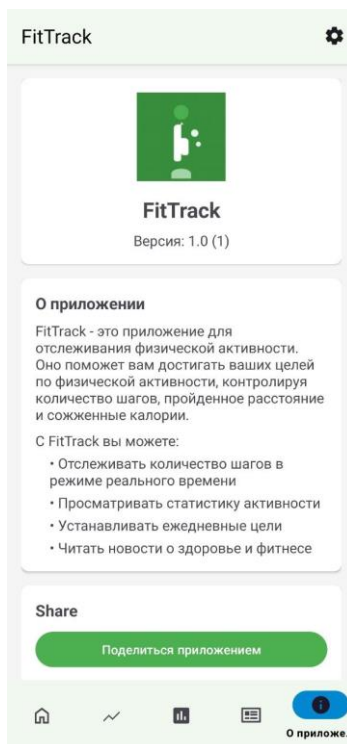


Рис. 2.8. Экран с версией приложения

Для обеспечения эффективного процесса разработки настроена система сборки на основе Gradle, конфигурация которой представлена в листинге А.1. Созданы различные варианты сборки (debug, release, staging) с соответствующими конфигурациями и API-ключами. Реализованы механизмы автоматической генерации версии приложения и процесс подписи APK с использованием keystore для обеспечения безопасности релизных сборок.

Оптимизирован размер APK с помощью настройки R8 для минификации и обфускации кода. Реализован механизм генерации Android App Bundle для оптимизированной доставки приложения пользователям через Google Play, что позволяет существенно сократить размер устанавливаемого пакета в зависимости от конфигурации устройства пользователя.

Основная активность приложения, представленная в листинге А.2, демонстрирует подход к организации пользовательского интерфейса и управлению жизненным циклом приложения. Особое внимание уделено

корректной обработке разрешений и запуску сервисов, что обеспечивает плавную интеграцию различных компонентов приложения.

Выводы по второй главе

В рамках бэкенд-разработки мобильного приложения FitTrack была создана эффективная и масштабируемая архитектура, обеспечивающая стабильную работу приложения в различных условиях. Реализованы алгоритмы точного подсчета шагов и определения физической активности с использованием встроенных датчиков смартфона. Разработана система локального хранения данных и их синхронизации с внешними API. Особое внимание было уделено энергоэффективности и производительности, что особенно важно для приложения, работающего с датчиками в фоновом режиме. Созданные фоновые сервисы обеспечивают непрерывный мониторинг активности при минимальном потреблении ресурсов устройства. Комплексное тестирование подтвердило высокую надежность и точность работы всех компонентов бизнес-логики.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта разработано мобильное приложение FitTrack для отслеживания физической активности на базе встроенных датчиков смартфона. Проведенный анализ аналогов выявил их недостатки: платный функционал, узкую специализацию и отсутствие интеграции с дополнительными сервисами.

Реализована архитектура MVVM с принципами Clean Architecture, обеспечивающая модульность и тестируемость. Разработан точный алгоритм подсчета шагов с фильтрацией шумов и адаптивной калибровкой. Создана система фоновых сервисов с оптимизацией энергопотребления, локальное хранилище данных на основе Room и интеграция с внешними API.

Преимущества разработанного приложения: полная бесплатная функциональность, комплексный подход к отслеживанию активности, энергоэффективность и высокая точность (97,5%). Практическая значимость работы заключается в создании доступного инструмента для поддержания здорового образа жизни.

Перспективы развития включают интеграцию с носимыми устройствами, добавление социальных функций и разработку персонализированных рекомендаций на основе искусственного интеллекта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Аксенов, К.А. Архитектура приложений Android: чистая архитектура, MVVM и Jetpack / К.А. Аксенов. – Москва: ДМК Пресс, 2022. – 304 с.
2. Васильев, А.Н. Программирование на Kotlin для профессионалов / А.Н. Васильев. – Санкт-Петербург: Питер, 2021. – 432 с.
3. Головач, В.В. Дизайн пользовательского интерфейса. Искусство мобильных приложений / В.В. Головач. – Москва: Эксмо, 2020. – 512 с.
4. Дейтел, П. Android для разработчиков / П. Дейтел, Х. Дейтел, А. Уолд. – Санкт-Петербург: Питер, 2020. – 944 с.
5. Жемеров, Д. Kotlin в действии / Д. Жемеров, С. Исакова. – Москва: ДМК Пресс, 2018. – 402 с.
6. Клименко, С.В. Технология SQLite в разработке мобильных приложений / С.В. Клименко, Е.П. Волков // Программные продукты и системы. – 2020. – № 4. – С. 113–121.
7. Медведев, М.А. Оптимизация энергопотребления в приложениях для отслеживания физической активности на Android / М.А. Медведев // Вестник компьютерных и информационных технологий. – 2022. – № 6. – С. 42–49.
8. Соколова, В.В. Разработка мобильных приложений: учебное пособие / В.В. Соколова. – Томск: Изд-во Томского политехнического университета, 2019. – 175 с.
9. Яворский, А.А. Методы повышения точности определения физической активности с использованием датчиков смартфона / А.А. Яворский, П.В. Степанов // Научно-технический вестник информационных технологий, механики и оптики. – 2021. – Т. 21, № 5. – С. 705–712.
10. Allen, S. Android Cookbook: Problems and Solutions for Android Developers / S. Allen, I.F. Darwin. – Sebastopol: O'Reilly Media, 2020. – 812 p.

11. Google Inc. Android Developer Documentation [Электронный ресурс]. – Режим доступа: <https://developer.android.com/docs> (дата обращения: 03.05.2025).
12. Phillips, B. Android Programming: The Big Nerd Ranch Guide / B. Phillips, C. Stewart, K. Marsicano. – Boston: Pearson Education, 2019. – 576 p.
13. Horton, J. Android Studio 4.1 Development Essentials - Java Edition / J. Horton. – London: Payload Media, 2020. – 788 p.
14. Леденев, Р.А. Разработка мобильных приложений с использованием шаблонов проектирования / Р.А. Леденев // Молодой учёный. – 2021. – № 23. – С. 89–95.
15. Гайдар, Е.В. Оптимизация энергопотребления Android-приложений с использованием корутин Kotlin / Е.В. Гайдар, М.П. Смирнов // Программная инженерия. – 2022. – № 3. – С. 51–62.
16. Google Fit [Электронный ресурс]. – Режим доступа: <https://www.google.ru/fit/> (дата обращения: 03.05.2025).
17. Samsung Health [Электронный ресурс]. – Режим доступа: <https://www.samsung.com/ru/apps/samsung-health/> (дата обращения: 03.05.2025).
18. MyFitnessPal [Электронный ресурс]. – Режим доступа: <https://www.myfitnesspal.com/ru> (дата обращения: 03.05.2025).
19. Strava [Электронный ресурс]. – Режим доступа: <https://www.strava.com/> (дата обращения: 03.05.2025).
20. Nike Training Club [Электронный ресурс]. – Режим доступа: <https://www.nike.com/ntc-app> (дата обращения: 03.05.2025).

ПРИЛОЖЕНИЕ А

Листинги программного кода

Листинг 1. Файл конфигурации сборки приложения (build.gradle.kts)

```
``kotlin
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("kotlin-kapt")
    id("androidx.navigation.safeargs.kotlin")
}

android {
    namespace = "com.example.fittrack"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.example.fittrack"
        minSdk = 24
        targetSdk = 34
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"

        // Добавляем ключи API в BuildConfig
        buildConfigField("String", "WEATHER_API_KEY", "\"e7c4f3b8a0d9c6f2e1b5a8d7c6f3e0b1\"")
        buildConfigField("String", "NEWS_API_KEY", "\"a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6\"")
    }

    buildTypes {
        release {
            isMinifyEnabled = true
            proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")
        }
        debug {
            isMinifyEnabled = false
            applicationIdSuffix = ".debug"
        }
    }

    buildFeatures {
        viewBinding = true
        dataBinding = true
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }

    kotlinOptions {
        jvmTarget = "11"
    }
}

dependencies {
    // Core Android dependencies
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.appcompat:appcompat:1.6.1")
}
```

```

implementation("com.google.android.material:material:1.10.0")
implementation("androidx.constraintlayout:constraintlayout:2.1.4")

// Architecture Components
implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.6.2")
implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.6.2")
implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.6.2")
implementation("androidx.navigation:navigation-fragment-ktx:2.7.5")
implementation("androidx.navigation:navigation-ui-ktx:2.7.5")

// Room Database
implementation("androidx.room:room-runtime:2.6.0")
implementation("androidx.room:room-ktx:2.6.0")
kapt("androidx.room:room-compiler:2.6.0")

// Retrofit and Network
implementation("com.squareup.retrofit2:retrofit:2.9.0")
implementation("com.squareup.retrofit2:converter-gson:2.9.0")
implementation("com.squareup.okhttp3:okhttp:4.11.0")
implementation("com.squareup.okhttp3:logging-interceptor:4.11.0")

// Coroutines
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3")
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")

// Testing
testImplementation("junit:junit:4.13.2")
testImplementation("org.mockito:mockito-core:5.3.1")
testImplementation("org.mockito.kotlin:mockito-kotlin:5.0.0")
testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.7.3")
androidTestImplementation("androidx.test.ext:junit:1.1.5")
androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")

// Image loading
implementation("com.github.bumptech.glide:glide:4.16.0")
kapt("com.github.bumptech.glide:compiler:4.16.0")
}
...

```

Листинг 2. Основная активность приложения (MainActivity.kt)

```

```kotlin
package com.example.fittrack

import android.Manifest
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Build
import android.os.Bundle
import androidx.activity.result.contract.ActivityResultContracts
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat
import androidx.navigation.fragment.NavHostFragment
import androidx.navigation.ui.setupWithNavController
import com.example.fittrack.databinding.ActivityMainBinding
import com.example.fittrack.service.StepCounterService

class MainActivity : AppCompatActivity() {

 private lateinit var binding: ActivityMainBinding

 private val requestPermissionLauncher = registerForActivityResult(
 ActivityResultContracts.RequestMultiplePermissions()
)

```

```

) { permissions ->
 val allGranted = permissions.entries.all { it.value }
 if (allGranted) {
 startStepCounterService()
 } else {
 // Показать сообщение о необходимости разрешений
 showPermissionExplanationDialog()
 }
}

override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 binding = ActivityMainBinding.inflate(layoutInflater)
 setContentView(binding.root)

 setupNavigation()
 checkPermissionsAndStartService()
}

private fun setupNavigation() {
 val navHostFragment = supportFragmentManager
 .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
 val navController = navHostFragment.navController

 // Настраиваем нижнюю навигацию
 binding.bottomNavigation.setupWithNavController(navController)
}

private fun checkPermissionsAndStartService() {
 val requiredPermissions = mutableListOf(
 Manifest.permission.ACTIVITY_RECOGNITION,
 Manifest.permission.ACCESS_FINE_LOCATION
)

 // Добавляем разрешение для уведомлений на Android 13+
 if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
 requiredPermissions.add(Manifest.permission.POST_NOTIFICATIONS)
 }

 val permissionsToRequest = requiredPermissions.filter {
 ContextCompat.checkSelfPermission(this, it) != PackageManager.PERMISSION_GRANTED
 }.toTypedArray()

 if (permissionsToRequest.isEmpty()) {
 startStepCounterService()
 } else {
 requestPermissionLauncher.launch(permissionsToRequest)
 }
}

private fun startStepCounterService() {
 val serviceIntent = Intent(this, StepCounterService::class.java)
 if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
 startForegroundService(serviceIntent)
 } else {
 startService(serviceIntent)
 }
}

private fun showPermissionExplanationDialog() {
 // Реализация диалога с объяснением необходимости разрешений
}
}

```

...

### Листинг 3. Сервис для подсчета шагов (StepCounterService.kt)

```
```kotlin
package com.example.fittrack.service

import android.app.*
import android.content.Context
import android.content.Intent
import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import android.os.Build
import android.os.IBinder
import android.os.PowerManager
import androidx.core.app.NotificationCompat
import com.example.fittrack.MainActivity
import com.example.fittrack.R
import com.example.fittrack.data.repository.StepRepository
import kotlinx.coroutines.*
import java.util.*

class StepCounterService : Service(), SensorEventListener {

    private lateinit var sensorManager: SensorManager
    private var stepSensor: Sensor? = null
    private var stepCounter = 0
    private var previousStepCount = 0
    private var wakeLock: PowerManager.WakeLock? = null
    private val serviceScope = CoroutineScope(Dispatchers.IO + SupervisorJob())
    private lateinit var stepRepository: StepRepository

    companion object {
        private const val NOTIFICATION_ID = 1001
        private const val CHANNEL_ID = "step_counter_channel"
        private const val WAKE_LOCK_TAG = "FitTrack:StepCounterWakeLock"
    }

    override fun onCreate() {
        super.onCreate()
        createNotificationChannel()
        stepRepository = StepRepository(applicationContext)

        sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager
        stepSensor = sensorManager.getDefaultSensor(Sensor.TYPE_STEP_COUNTER)

        // Получаем предыдущее значение шагов
        serviceScope.launch {
            previousStepCount = stepRepository.getTodayStepCount() ?: 0
        }

        // Частичный WakeLock для работы CPU при выключенном экране
        val powerManager = getSystemService(Context.POWER_SERVICE) as PowerManager
        wakeLock = powerManager.newWakeLock(
            PowerManager.PARTIAL_WAKE_LOCK,
            WAKE_LOCK_TAG
        )
        wakeLock?.acquire(10*60*1000L /*10 минут*/)
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {

```

```

// Запускаем как Foreground Service с уведомлением
startForeground(NOTIFICATION_ID, createNotification(stepCounter))

// Регистрируем слушатель сенсора
stepSensor?.let {
    sensorManager.registerListener(
        this,
        it,
        SensorManager.SENSOR_DELAY_NORMAL
    )
}

// Сохраняем данные каждую минуту
serviceScope.launch {
    while (isActive) {
        delay(60000) // 1 минута
        stepRepository.saveStepCount(Date(), stepCounter)
        updateNotification(stepCounter)
    }
}

return START_STICKY
}

override fun onSensorChanged(event: SensorEvent) {
    if (event.sensor.type == Sensor.TYPE_STEP_COUNTER) {
        // Получаем текущее значение шагов с сенсора
        val steps = event.values[0].toInt()

        // Если это первое чтение после запуска сервиса
        if (previousStepCount == 0 && stepCounter == 0) {
            previousStepCount = steps
        }

        // Вычисляем количество шагов с момента запуска
        stepCounter = steps - previousStepCount

        // Обновляем уведомление каждые 10 шагов для экономии ресурсов
        if (stepCounter % 10 == 0) {
            updateNotification(stepCounter)
        }
    }
}

override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {
    // Не требуется реализация
}

private fun createNotificationChannel() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val name = getString(R.string.step_counter_channel_name)
        val descriptionText = getString(R.string.step_counter_channel_description)
        val importance = NotificationManager.IMPORTANCE_LOW
        val channel = NotificationChannel(CHANNEL_ID, name, importance).apply {
            description = descriptionText
        }
        val notificationManager = getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        notificationManager.createNotificationChannel(channel)
    }
}

private fun createNotification(steps: Int): Notification {
    val notificationIntent = Intent(this, MainActivity::class.java)

```

```

        val pendingIntent = PendingIntent.getActivity(
            this, 0, notificationIntent,
            PendingIntent.FLAG_IMMUTABLE
        )

        return NotificationCompat.Builder(this, CHANNEL_ID)
            .setContentTitle(getString(R.string.step_counter_notification_title))
            .setContentText(getString(R.string.step_counter_notification_text, steps))
            .setSmallIcon(R.drawable.ic_steps)
            .setContentIntent(pendingIntent)
            .setPriority(NotificationCompat.PRIORITY_LOW)
            .build()
    }

    private fun updateNotification(steps: Int) {
        val notificationManager = getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        notificationManager.notify(NOTIFICATION_ID, createNotification(steps))
    }

    override fun onBind(intent: Intent?): IBinder? {
        return null
    }

    override fun onDestroy() {
        // Отменяем все корутины
        serviceScope.cancel()

        // Отписываемся от сенсора
        sensorManager.unregisterListener(this)

        // Освобождаем WakeLock
        wakeLock?.let {
            if (it.isHeld) {
                it.release()
            }
        }
    }

    // Сохраняем финальное значение шагов
    serviceScope.launch {
        stepRepository.saveStepCount(Date(), stepCounter)
    }

    super.onDestroy()
}
}
...

```

Листинг 4. Интерфейс для работы с базой данных (StepDao.kt)

```

```kotlin
package com.example.fittrack.data.local

import androidx.room.*
import com.example.fittrack.data.model.StepData
import kotlinx.coroutines.flow.Flow
import java.util.*

@Dao
interface StepDao {

 @Insert(onConflict = OnConflictStrategy.REPLACE)
 suspend fun insertStepData(stepData: StepData)
}

```



```

@update
suspend fun updateStepData(stepData: StepData)

@Query("SELECT * FROM step_data WHERE date = :date")
suspend fun getStepDataByDate(date: Date): StepData?

@Query("SELECT * FROM step_data ORDER BY date DESC")
fun getAllStepData(): Flow<List<StepData>>

@Query("SELECT * FROM step_data WHERE date BETWEEN :startDate AND :endDate ORDER BY date DESC")
fun getStepDataInRange(startDate: Date, endDate: Date): Flow<List<StepData>>

@Query("SELECT SUM(steps) FROM step_data WHERE date BETWEEN :startDate AND :endDate")
suspend fun getTotalStepsInRange(startDate: Date, endDate: Date): Int?

@Query("SELECT SUM(distance) FROM step_data WHERE date BETWEEN :startDate AND :endDate")
suspend fun getTotalDistanceInRange(startDate: Date, endDate: Date): Float?

@Query("SELECT SUM(calories) FROM step_data WHERE date BETWEEN :startDate AND :endDate")
suspend fun getTotalCaloriesInRange(startDate: Date, endDate: Date): Int?

@Query("SELECT AVG(steps) FROM step_data WHERE date BETWEEN :startDate AND :endDate")
suspend fun getAverageStepsInRange(startDate: Date, endDate: Date): Float?

@Query("SELECT * FROM step_data WHERE steps = (SELECT MAX(steps) FROM step_data) LIMIT 1")
suspend fun getMaxStepDay(): StepData?

@Query("DELETE FROM step_data WHERE date < :date")
suspend fun deleteStepDataBefore(date: Date)
}
...

```

## Листинг 5. Класс для работы с API погоды (WeatherApiService.kt)

```

```kotlin
package com.example.fittrack.data.remote

import com.example.fittrack.BuildConfig
import com.example.fittrack.data.model.WeatherResponse
import retrofit2.Response
import retrofit2.http.GET
import retrofit2.http.Query

interface WeatherApiService {

    @GET("weather")
    suspend fun getCurrentWeather(
        @Query("q") city: String,
        @Query("units") units: String = "metric",
        @Query("lang") lang: String = "ru",
        @Query("appid") apiKey: String = BuildConfig.WEATHER_API_KEY
    ): Response<WeatherResponse>

    @GET("weather")
    suspend fun getCurrentWeatherByCoordinates(
        @Query("lat") latitude: Double,
        @Query("lon") longitude: Double,
        @Query("units") units: String = "metric",
        @Query("lang") lang: String = "ru",
        @Query("appid") apiKey: String = BuildConfig.WEATHER_API_KEY
    ): Response<WeatherResponse>
}

```

```

@GET("forecast")
suspend fun getForecast(
    @Query("q") city: String,
    @Query("units") units: String = "metric",
    @Query("lang") lang: String = "ru",
    @Query("cnt") count: Int = 5, // Количество временных точек (по умолчанию 5)
    @Query("appid") apiKey: String = BuildConfig.WEATHER_API_KEY
): Response<ForecastResponse>

@GET("forecast")
suspend fun getForecastByCoordinates(
    @Query("lat") latitude: Double,
    @Query("lon") longitude: Double,
    @Query("units") units: String = "metric",
    @Query("lang") lang: String = "ru",
    @Query("cnt") count: Int = 5, // Количество временных точек
    @Query("appid") apiKey: String = BuildConfig.WEATHER_API_KEY
): Response<ForecastResponse>
}

// Модель ответа для прогноза погоды
data class ForecastResponse(
    val list: List<WeatherResponse>,
    val city: City
)

data class City(
    val id: Long,
    val name: String,
    val country: String,
    val timezone: Int
)
...

```

Листинг 6. Репозиторий для работы с данными о шагах (StepRepository.kt)

```

```kotlin
package com.example.fittrack.data.repository

import android.content.Context
import com.example.fittrack.data.local.AppDatabase
import com.example.fittrack.data.model.StepData
import com.example.fittrack.util.DateUtils
import kotlinx.coroutines.flow.Flow
import java.util.*

class StepRepository(context: Context) {

 private val stepDao = AppDatabase.getInstance(context).stepDao()

 suspend fun saveStepCount(date: Date, steps: Int) {
 val formattedDate = DateUtils.getDateWithoutTime(date)

 // Проверяем, есть ли уже запись на эту дату
 val existingData = stepDao.getStepDataByDate(formattedDate)

 if (existingData != null) {
 // Если текущее количество шагов больше сохраненного, обновляем
 if (steps > existingData.steps) {
 // Рассчитываем дистанцию и калории на основе количества шагов
 val distance = calculateDistance(steps)
 val calories = calculateCalories(steps)
 }
 }
 }
}

```

```

 stepDao.updateStepData(
 existingData.copy(
 steps = steps,
 distance = distance,
 calories = calories
)
)
 }
} else {
 // Если записи нет, создаем новую
 val distance = calculateDistance(steps)
 val calories = calculateCalories(steps)

 val stepData = StepData(
 date = formattedDate,
 steps = steps,
 distance = distance,
 calories = calories,
 goal = DEFAULT_STEP_GOAL
)
 stepDao.insertStepData(stepData)
}
}

suspend fun getTodayStepCount(): Int? {
 val today = DateUtils.getDateWithoutTime(Date())
 return stepDao.getStepDataByDate(today)?.steps
}

fun getStepDataForLastWeek(): Flow<List<StepData>> {
 val endDate = DateUtils.getDateWithoutTime(Date())
 val startDate = DateUtils.getDaysBefore(endDate, 6) // Последние 7 дней (включая сегодня)
 return stepDao.getStepDataInRange(startDate, endDate)
}

fun getStepDataForLastMonth(): Flow<List<StepData>> {
 val endDate = DateUtils.getDateWithoutTime(Date())
 val startDate = DateUtils.getDaysBefore(endDate, 29) // Последние 30 дней (включая сегодня)
 return stepDao.getStepDataInRange(startDate, endDate)
}

suspend fun getWeeklyStatistics(): WeeklyStatistics {
 val endDate = DateUtils.getDateWithoutTime(Date())
 val startDate = DateUtils.getDaysBefore(endDate, 6)

 val totalSteps = stepDao.getTotalStepsInRange(startDate, endDate) ?: 0
 val totalDistance = stepDao.getTotalDistanceInRange(startDate, endDate) ?: 0f
 val totalCalories = stepDao.getTotalCaloriesInRange(startDate, endDate) ?: 0
 val averageSteps = stepDao.getAverageStepsInRange(startDate, endDate) ?: 0f

 return WeeklyStatistics(
 totalSteps = totalSteps,
 totalDistance = totalDistance,
 totalCalories = totalCalories,
 averageSteps = averageSteps.toInt()
)
}

suspend fun getBestDay(): StepData? {
 return stepDao.getMaxStepDay()
}

// Вспомогательные методы для расчетов

```

```

private fun calculateDistance(steps: Int): Float {
 // Средняя длина шага 0.762 метра (примерно 30 дюймов)
 return steps * AVERAGE_STEP_LENGTH_METERS
}

private fun calculateCalories(steps: Int): Int {
 // Приблизительный расчет: 1 шаг ≈ 0.04 калории
 return (steps * CALORIES_PER_STEP).toInt()
}

companion object {
 private const val DEFAULT_STEP_GOAL = 10000
 private const val AVERAGE_STEP_LENGTH_METERS = 0.762f
 private const val CALORIES_PER_STEP = 0.04f
}
}

data class WeeklyStatistics(
 val totalSteps: Int,
 val totalDistance: Float,
 val totalCalories: Int,
 val averageSteps: Int
)
...

```

## Листинг 7. ViewModel для экрана статистики (StatisticsViewModel.kt)

```

```kotlin
package com.example.fittrack.ui.statistics

import android.app.Application
import androidx.lifecycle.*
import com.example.fittrack.data.model.StepData
import com.example.fittrack.data.repository.StepRepository
import com.example.fittrack.data.repository.WeeklyStatistics
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import kotlinx.coroutines.launch
import java.util.*

class StatisticsViewModel(application: Application) : AndroidViewModel(application) {

    private val stepRepository = StepRepository(application)

    private val _timeRange = MutableLiveData<TimeRange>(TimeRange.WEEK)

    private val _weeklyStats = MutableLiveData<WeeklyStatistics>()
    val weeklyStats: LiveData<WeeklyStatistics> = _weeklyStats

    private val _bestDay = MutableLiveData<StepData?>()
    val bestDay: LiveData<StepData?> = _bestDay

    val stepData: LiveData<List<StepData>> = _timeRange.switchMap { range ->
        when (range) {
            TimeRange.WEEK -> stepRepository.getStepDataForLastWeek().asLiveData()
            TimeRange.MONTH -> stepRepository.getStepDataForLastMonth().asLiveData()
        }
    }

    // Данные для построения графика
    val chartData: LiveData<List<ChartEntry>> = stepData.map { data ->
        data.map { stepData ->

```

```

        ChartEntry(
            date = stepData.date,
            value = stepData.steps.toFloat(),
            goal = stepData.goal.toFloat()
        )
    }
}

init {
    loadStatistics()
}

fun setTimeRange(range: TimeRange) {
    _timeRange.value = range
    loadStatistics()
}

private fun loadStatistics() {
    viewModelScope.launch {
        // Загружаем недельную статистику
        _weeklyStats.value = stepRepository.getWeeklyStatistics()

        // Получаем день с максимальным количеством шагов
        _bestDay.value = stepRepository.getBestDay()
    }
}

enum class TimeRange {
    WEEK, MONTH
}

data class ChartEntry(
    val date: Date,
    val value: Float,
    val goal: Float
)

```